

REACT

MEDIUM

Build a useToggle Hook

Your team keeps writing the same three lines (a boolean piece of state plus a handler that flips it) in every component that has a show/hide button, an accordion, or a modal. The lead engineer asks you to extract this into a small reusable custom hook so it can be imported anywhere instead of re-implemented.

```
function useToggle(initialValue?: boolean) {
  // TODO: implement this hook
}

function ToggleDemo() {
  const [isOn, toggle] = useToggle(false);
  return <button onClick={toggle}>{isOn ? "ON" : "OFF"}</button>;
}
```

Your task:

1. Implement useToggle(initialValue) so it returns a tuple [value, toggle] where toggle flips the boolean value.
2. Make sure the toggle function does not become a new function identity on every render (it should be stable across re-renders).
3. Give the hook and its return value correct TypeScript types (no implicit any).

MEDIUM

Fix the Stale Closure Counter

A junior developer built a live counter that should tick up once per second, but QA reports it only ever goes from 0 to 1 and then freezes. The code below is what they wrote.

```
function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const id = setInterval(() => {
      setCount(count + 1);
    }, 1000);
    return () => clearInterval(id);
  }, []);

  return <p>{count}</p>;
}
```

Your task:

1. Explain in one or two sentences why the counter stops at 1.
2. Fix the code so the counter keeps incrementing every second, without adding count to the dependency array (that would re-create the interval every tick).

REACT

MEDIUM

Controlled Signup Form Validation

Build a small signup form component with an email field and a password field. It should validate as the user types: the email must look like a valid address, and the password must be at least 8 characters. The submit button should be disabled while the form is invalid, and each invalid field should show its own inline error message.

```
interface FormState {
  email: string;
  password: string;
}

function SignupForm() {
  // TODO: implement controlled state, validation, and submit handling
  return null;
}
```

Your task:

1. Implement SignupForm as a fully controlled component using a single FormState object in useState.
2. Write a validate(values) function that returns an errors object with optional email/password messages.
3. Render an inline error message next to each invalid field, and disable the submit button while any errors exist.
4. Handle form submission by preventing the default page reload.

REACT

MEDIUM

Fix Index as Key Bug

A todo list lets users check off items and also supports drag-to-reorder (implemented elsewhere, not shown). Testers noticed that after reordering the list, the checkboxes appear checked next to the wrong todo text. The component below is the suspected cause.

```
function TodoList({ todos }: { todos: { id: string; text: string }[] }) {
  return (
    <ul>
      {todos.map((todo, index) => (
        <li key={index}>
          <input type="checkbox" /> {todo.text}
        </li>
      ))}
    </ul>
  );
}
```

Your task:

1. Explain why using the array index as the key causes checkbox state to appear attached to the wrong item after reordering.
2. Fix the component so each list item is keyed stably regardless of position.

REACT

MEDIUM

Shopping Cart Context and Reducer

The app has cart-related UI scattered across several unrelated components (product cards, a mini-cart badge, a checkout summary), all of which need to read and update the same cart contents. Rather than passing cart state down through many props, build a CartContext backed by useReducer that any component can consume.

```
interface CartItem {
  id: string;
  name: string;
  quantity: number;
}

// TODO: define CartAction, cartReducer, CartContext, CartProvider, and useCart
```

Your task:

1. Define a CartAction union type supporting ADD_ITEM (adding one unit, or incrementing quantity if already present) and REMOVE_ITEM.
2. Write cartReducer(state, action) implementing both actions immutably.
3. Create CartContext plus a CartProvider component that wraps useReducer and provides { items, dispatch }.
4. Write a useCart() hook that reads the context and throws a clear error if used outside CartProvider.

REACT

MEDIUM

Memoize a Filtered Product List

A product list screen has an unrelated click counter in the corner (for a feature flag experiment). Every time the counter is clicked, the entire filtered product list visibly flickers and console logs show the memoized child list re-rendering even though the products and query haven't changed.

```
interface Product {
  id: string;
  name: string;
}

const MemoList = memo(function MemoList({ items }: { items: Product[] }) {
  return (
    <ul>
      {items.map((p) => (
        <li key={p.id}>{p.name}</li>
      ))}
    </ul>
  );
});

function ProductList({ products, query }: { products: Product[]; query: string }) {
  const [count, setCount] = useState(0);

  const filtered = products.filter((p) => p.name.includes(query));

  return (
    <div>
      <button onClick={() => setCount(count + 1)}>Clicks: {count}</button>
      <MemoList items={filtered} />
    </div>
  );
}
```

Your task:

1. Explain why MemoList re-renders on every click even though it is wrapped in memo().
2. Fix ProductList so the filtered array keeps the same reference across renders unless products or query actually change.

MEDIUM

Lift State Between Temperature Inputs

Design a small temperature calculator with two sibling inputs, Celsius and Fahrenheit. Typing in either input should immediately update the other so they always represent the same temperature. `TemperatureInput` is a presentational component that has already been written for you; you need to build the parent that lifts the shared state up.

```
type Scale = "c" | "f";

function TemperatureInput({
  scale,
  temperature,
  onTemperatureChange,
}: {
  scale: Scale;
  temperature: string;
  onTemperatureChange: (value: string) => void;
}) {
  return (
    <fieldset>
      <legend>{scale === "c" ? "Celsius" : "Fahrenheit"}</legend>
      <input value={temperature} onChange={(e) => onTemperatureChange(e.target.value)} />
    </fieldset>
  );
}

// TODO: implement Calculator, which renders two TemperatureInputs kept in sync
```

Your task:

1. Write helper functions `toCelsius`, `toFahrenheit`, and `tryConvert(temperature, convert)` that returns an empty string for invalid numeric input.
2. Implement `Calculator`, holding the shared temperature value and which scale was last edited in state at the parent level.
3. Pass each `TemperatureInput` its own derived value (converting when it is not the source of truth) and an `onTemperatureChange` handler that updates the lifted state.

MEDIUM

Fix Conditional Rendering Zero Bug

A cart summary widget is supposed to show either the list of items or an 'empty cart' message. Instead, when the cart is empty, users see a stray '0' rendered on the page.

```
function CartSummary({ items }: { items: string[] }) {
  return (
    <div>
      {items.length} && <ul>{items.map((i) => <li key={i}>{i}</li>)}</ul>
    </div>
  );
}
```

Your task:

1. Explain why an empty items array causes the literal text '0' to render.
2. Fix CartSummary so it shows the list when there are items and an 'Your cart is empty' message otherwise, with no stray output.

MEDIUM

Build a Compound Tabs Component

Design a reusable Tabs component using the compound component pattern so consumers can write markup like `<Tabs><Tabs.List><Tabs.Tab index={0}>...</Tabs.Tab></Tabs.List><Tabs.Panel index={0}>...</Tabs.Panel></Tabs>`, without Tabs needing to know its children's exact structure in advance. The active tab index should be shared implicitly between the sub-components.

```
// TODO: implement Tabs, Tabs.List, Tabs.Tab, and Tabs.Panel
// so that clicking a Tab shows the matching Panel and hides the others.
```

Your task:

1. Share the active tab index between sub-components using React context, not prop drilling.
2. Implement Tab so clicking it sets the active index, and give it an aria-selected attribute reflecting whether it's active.
3. Implement Panel so it renders its children only when its index matches the active index.
4. Attach List, Tab, and Panel onto Tabs as Tabs.List, Tabs.Tab, Tabs.Panel.

REACT

MEDIUM

Build a useDebounce Hook

A search box fires an expensive network request on every keystroke, overwhelming the backend. Product wants the search to only fire 300ms after the user stops typing. Extract a generic useDebounce hook that can be reused for any value, not just strings.

```
function useDebounce<T>(value: T, delay: number): T {
  // TODO: implement
}

function SearchBox() {
  const [query, setQuery] = useState("");
  const debouncedQuery = useDebounce(query, 300);
  // TODO: trigger the search whenever debouncedQuery changes
  return <input value={query} onChange={(e) => setQuery(e.target.value)} />;
}
```

Your task:

1. Implement useDebounce(value, delay) so it returns a debounced copy of value that only updates after the value has been stable for delay milliseconds.
2. Make sure a pending timeout from a previous render is cancelled if value changes again before delay elapses.
3. Wire SearchBox to log/search using debouncedQuery whenever it changes.

MEDIUM

Fix Stale Threshold in Resize Listener

A component logs a warning whenever the window is resized below a user-adjustable threshold. QA reports that after changing the threshold with the slider, the warning still uses the old threshold value indefinitely.

```
function WindowWidthLogger() {
  const [width, setWidth] = useState(window.innerWidth);
  const [threshold, setThreshold] = useState(768);

  useEffect(() => {
    function handleResize() {
      if (window.innerWidth < threshold) {
        console.log("below threshold");
      }
      setWidth(window.innerWidth);
    }
    window.addEventListener("resize", handleResize);
    return () => window.removeEventListener("resize", handleResize);
  }, []);

  return <p>{width}</p>;
}
```

Your task:

1. Explain why `handleResize` keeps using the threshold value from the very first render.
2. Fix the effect so resizing always compares against the current threshold.

REACT

MEDIUM

Fix Uncontrolled Input Warning

An email input field works fine while typing, but the browser console shows: 'A component is changing an uncontrolled input to be controlled.' This happens the moment the user types their first character.

```
function EmailField() {
  const [email, setEmail] = useState();

  return (
    <input value={email} onChange={(e) => setEmail(e.target.value)} />
  );
}
```

Your task:

1. Explain why the input starts out uncontrolled and then switches to controlled.
2. Fix EmailField so the input is controlled from the very first render, with correct TypeScript typing for the state.

MEDIUM

Build a Filterable List Component

Build a small component that takes a flat list of string items and a search box above it. Typing in the box should filter the visible items in real time (case-insensitive substring match), and the component should show a friendly message when nothing matches.

```
function FilterableList({ items }: { items: string[] }) {
  // TODO: implement search input + filtered rendering
  return null;
}
```

Your task:

1. Implement a controlled search input backed by `useState`.
2. Filter items case-insensitively based on the current query on every render.
3. Render each match in a `<ul>` with a stable, warning-free key, and show a 'No results' message when the filtered list is empty.

REACT

MEDIUM

Multi Step Wizard with useReducer

Build a 3-step signup wizard. Users move forward and backward between steps, and data entered on earlier steps (name, then email) must be preserved when navigating. Model this with useReducer instead of several separate useState calls, since the step and the data update together.

```
interface WizardState {
  step: number;
  data: { name: string; email: string };
}

type WizardAction =
  | { type: "NEXT" }
  | { type: "BACK" }
  | { type: "UPDATE_FIELD"; field: keyof WizardState["data"]; value: string };

// TODO: implement wizardReducer and the Wizard component
```

Your task:

1. Implement wizardReducer handling NEXT (clamped at step 3), BACK (clamped at step 1), and UPDATE_FIELD (updates one field of data immutably).
2. Implement Wizard, showing the name input on step 1, the email input on step 2, and Back/Next buttons that are disabled at the boundaries.

MEDIUM

Fix Broken Memoized Button

A toolbar has an expensive Save button (memoized with `React.memo`, since it does costly layout work) sitting next to an unrelated click counter. Every time the counter increments, console logs show the Save button re-rendering, which shouldn't happen since neither its label nor its behavior changed.

```
const ExpensiveButton = memo(function ExpensiveButton({
  onClick,
  label,
}): {
  onClick: () => void;
  label: string;
}) {
  console.log("rendering", label);
  return <button onClick={onClick}>{label}</button>;
});

function Toolbar() {
  const [count, setCount] = useState(0);

  function handleSave() {
    console.log("saved");
  }

  return (
    <div>
      <p>{count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <ExpensiveButton onClick={handleSave} label="Save" />
    </div>
  );
}
```

Your task:

1. Explain why `memo()` on `ExpensiveButton` fails to prevent the re-render here.
2. Fix `Toolbar` so `ExpensiveButton` only re-renders when its own props actually change.

MEDIUM

Build a usePrevious Hook

A price ticker component needs to show whether the price went up, down, or stayed the same compared to its previous value. Build a generic `usePrevious` hook that returns whatever value a piece of state held on the previous render (undefined on the very first render).

```
function usePrevious<T>(value: T): T | undefined {
  // TODO: implement
}

function PriceTracker({ price }: { price: number }) {
  const previousPrice = usePrevious(price);
  // TODO: derive direction: "up" | "down" | "same" | "initial"
  return <p>{price}</p>;
}
```

Your task:

1. Implement `usePrevious(value)` using a `ref` so it returns the value from the previous render without causing an extra re-render itself.
2. Use it in `PriceTracker` to compute and display whether the price moved up, down, stayed the same, or this is the initial render.

MEDIUM

Mouse Tracker Render Prop

Different parts of the app need to react to mouse position in completely different ways (one shows coordinates as text, another moves an image). Rather than building a separate component for each visualization, implement a `MouseTracker` component that tracks mouse position internally and hands it off via a render prop, letting each consumer decide what to render.

```
interface Point {
  x: number;
  y: number;
}

function MouseTracker({ render }: { render: (point: Point) => ReactNode }) {
  // TODO: track mouse position on mousemove and call render(point)
  return null;
}
```

Your task:

1. Track the current mouse position in state, updated via an `onMouseMove` handler on a wrapping `<div>`.
2. Call the render prop with the current point and render its result.
3. Write a small example usage that renders the coordinates as text.

MEDIUM

Fix a Fetch Race Condition

A user profile panel fetches user data whenever `userId` changes (e.g. as the user clicks through a list of teammates quickly). QA reports that if they click two different users in fast succession, the panel sometimes ends up showing the wrong person's data, because an older, slower request resolves after a newer, faster one.

```
interface User {
  id: string;
  name: string;
}

declare function fetchUser(id: string): Promise<User>;

function UserProfile({ userId }: { userId: string }) {
  const [user, setUser] = useState<User | null>(null);

  useEffect(() => {
    fetchUser(userId).then((data) => {
      setUser(data);
    });
  }, [userId]);

  return <p>{user ? user.name : "Loading..."}</p>;
}
```

Your task:

1. Explain why a stale in-flight request can overwrite fresher data here.
2. Fix the effect so a response from an outdated request is ignored once a newer `userId` has been requested.

MEDIUM

Fix Mutated Checkbox Group State

A notification preferences form lets a user check any combination of email/sms/push. Testers report that checking or unchecking a box sometimes doesn't visually update at all, and sometimes the wrong box appears to toggle.

```
function PreferencesForm() {
  const options = ["email", "sms", "push"];
  const [selected, setSelected] = useState<string[]>([]);

  function handleChange(option: string) {
    selected.push(option);
    setSelected(selected);
  }

  return (
    <div>
      {options.map((option) => (
        <label key={option}>
          <input
            type="checkbox"
            checked={selected.includes(option)}
            onChange={() => handleChange(option)}
          />
          {option}
        </label>
      ))}
    </div>
  );
}
```

Your task:

1. Explain the two separate bugs in `handleChange`: why unchecking never works, and why React sometimes fails to re-render at all.
2. Fix `handleChange` so it correctly adds an option when unchecked and removes it when checked, using immutable state updates.

REACT

MEDIUM

Fix Mutated Shopping List State

A shopping list lets users type an item and click Add. Testers report that clicking Add sometimes appears to do nothing — the item doesn't show up in the list until some unrelated re-render happens later.

```
function ShoppingList() {
  const [items, setItems] = useState<string[]>(["Milk"]);
  const [text, setText] = useState("");

  function addItem() {
    items.push(text);
    setItems(items);
    setText("");
  }

  return (
    <div>
      <input value={text} onChange={(e) => setText(e.target.value)} />
      <button onClick={addItem}>Add</button>
      <ul>
        {items.map((item, i) => (
          <li key={i}>{item}</li>
        ))}
      </ul>
    </div>
  );
}
```

Your task:

1. Explain why calling `setItems(items)` after `items.push(text)` fails to trigger a re-render.
2. Fix `addItem` so adding an item reliably updates the rendered list.

REACT

MEDIUM

Theme Toggle with Context

The app needs a light/dark theme that any deeply nested component can read and toggle, without threading a theme prop through every layer of the component tree. Build a ThemeContext with a provider and a custom hook for consuming it.

```
type Theme = "light" | "dark";

// TODO: implement ThemeContext, ThemeProvider, useTheme, and ThemeToggleButton
```

Your task:

1. Create a ThemeContext whose value includes the current theme and a function to toggle it.
2. Implement ThemeProvider, holding the theme in state and wrapping children with the context provider.
3. Implement a useTheme() hook that throws if called outside ThemeProvider.
4. Implement ThemeToggleButton, a consumer that displays the current theme and toggles it on click.

REACT

MEDIUM

Memoize an Expensive Total

An analytics panel recomputes an order total from a large orders array. It also has an unrelated text filter input above it. Every keystroke in the filter box visibly lags because the console shows 'computing total...' logging on every single render, even though orders hasn't changed.

```
interface Order {
  id: string;
  amount: number;
}

function AnalyticsPanel({ orders }: { orders: Order[] }) {
  const [filter, setFilter] = useState("");

  function computeTotal(orders: Order[]): number {
    console.log("computing total...");
    return orders.reduce((sum, o) => sum + o.amount, 0);
  }

  const total = computeTotal(orders);

  return (
    <div>
      <input value={filter} onChange={(e) => setFilter(e.target.value)} />
      <p>Total: {total}</p>
    </div>
  );
}
```

Your task:

1. Explain why computeTotal runs on every keystroke even though it has nothing to do with the filter input.
2. Fix AnalyticsPanel so the total is only recomputed when orders actually changes.

MEDIUM

Build a useLocalStorage Hook

Several settings fields in the app (display name, preferred units, etc.) should persist to `localStorage` automatically and rehydrate on page load. Rather than duplicating the read/write logic in each component, build a generic `useLocalStorage` hook that behaves like `useState` but is backed by `localStorage`.

```
function useLocalStorage<T>(key: string, initialValue: T): [T, (value: T) => void] {
  // TODO: implement
}

function NameField() {
  const [name, setName] = useLocalStorage<string>("name", "");
  return <input value={name} onChange={e => setName(e.target.value)} />;
}
```

Your task:

1. Implement `useLocalStorage` so its initial value is read from `localStorage` if present (falling back to `initialValue`, and to `initialValue` if parsing fails).
2. Persist the value to `localStorage` whenever it changes.
3. Match the `useState`-like API: `[value, setValue]`.

MEDIUM

Loading Error Success State Machine

A posts list currently tracks loading with two separate booleans (`isLoading`, `isError`), which has already caused a bug where both were true at once and the UI showed conflicting content. Rewrite the state as a single discriminated union so illegal states (like loading and error simultaneously) are unrepresentable.

```
interface Post {
    id: string;
    title: string;
}

declare function fetchPosts(): Promise<Post[]>;

type RequestState<T> =
    | { status: "idle" }
    | { status: "loading" }
    | { status: "success"; data: T }
    | { status: "error"; error: string };

// TODO: implement PostsView using RequestState<Post[]>
```

Your task:

1. Implement `PostsView` using a single `useState<RequestState<Post[]>>` instead of separate boolean flags.
2. Kick off the fetch in an effect, transitioning through 'loading' then to 'success' or 'error'.
3. Render distinct UI for each of the four states, narrowing the type via `state.status` so TypeScript knows which fields are available in each branch.

REACT

MEDIUM

Compose a Card with Children

A generic Card, CardHeader, and CardBody have already been built as pure layout/slot components that just render whatever children they're given. Use composition (not new props on Card) to build a ProductCard that displays a product's title and description inside this existing layout.

```
function Card({ children }: { children: ReactNode }) {
  return <div className="card">{children}</div>;
}

function CardHeader({ children }: { children: ReactNode }) {
  return <div className="card-header">{children}</div>;
}

function CardBody({ children }: { children: ReactNode }) {
  return <div className="card-body">{children}</div>;
}

// TODO: implement ProductCard({ title, description }) using Card/CardHeader/CardBody
```

Your task:

1. Implement ProductCard({ title, description }) by composing Card, CardHeader, and CardBody, without modifying any of the three existing components.
2. Make sure title renders inside the header slot and description inside the body slot.

MEDIUM

Fix Missing Effect Dependency

A Greeting component receives a name prop from its parent (which changes as the user switches between team members in a dropdown elsewhere on the page). Testers notice the greeting text never updates after the very first name it was given, even though the prop clearly changes.

```
function Greeting({ name }: { name: string }) {
  const [greeting, setGreeting] = useState("");

  useEffect(() => {
    setGreeting(`Hello, ${name}!`);
  }, []);

  return <p>{greeting}</p>;
}
```

Your task:

1. Explain why the greeting text is frozen after the first render despite name changing.
2. Fix the component so the greeting always reflects the current name prop.

REACT

MEDIUM

Build a useOnClickOutside Hook

A dropdown menu should close whenever the user clicks anywhere outside of it. Extract this as a reusable useOnClickOutside hook that takes a ref to the element to watch and a handler to call on an outside click, so it can be reused for dropdowns, modals, and popovers alike.

```
function useOnClickOutside<T extends HTMLElement>(
  ref: RefObject<T | null>,
  handler: () => void
): void {
  // TODO: implement
}

function Dropdown() {
  const [open, setOpen] = useState(false);
  const ref = useRef<HTMLDivElement>(null);
  useOnClickOutside(ref, () => setOpen(false));

  return (
    <div ref={ref}>
      <button onClick={() => setOpen((o) => !o)}>Menu</button>
      {open && <ul><li>Option 1</li></ul>}
    </div>
  );
}
```

Your task:

1. Implement useOnClickOutside by attaching a document-level mousedown listener that checks whether the click target is contained within ref.current.
2. Call handler only when the click landed outside the referenced element, and skip entirely if ref.current is null.
3. Clean up the listener when the component unmounts or the ref/handler changes.

REACT

MEDIUM

Fix Key Bug in Contact Form

A contact editor renders one uncontrolled text input per contact (using `defaultValue`, since edits are only saved on a separate 'Save' action elsewhere). Testers report that after removing a contact from the middle of the list, one of the remaining inputs shows the wrong name — as if the text 'bled' from a different row.

```
function ContactForm({ contacts }: { contacts: { id: string; name: string }[] }) {
  const [localContacts, setLocalContacts] = useState(contacts);

  function removeContact(id: string) {
    setLocalContacts((prev) => prev.filter((c) => c.id !== id));
  }

  return (
    <div>
      {localContacts.map((contact, index) => (
        <div key={index}>
          <input defaultValue={contact.name} />
          <button onClick={() => removeContact(contact.id)}>Remove</button>
        </div>
      ))}
    </div>
  );
}
```

Your task:

1. Explain why removing a middle contact causes a remaining input to show the wrong name, specifically because these inputs are uncontrolled (`defaultValue`).
2. Fix the component so removing any contact never causes another input's displayed value to change.

REACT

MEDIUM

Lift State for Search Filtering

SearchBox (a controlled input) and UserList (a filtered list display) have already been implemented as independent, reusable, 'dumb' components that don't manage their own state. Build the parent SearchableUserList that lifts the shared query state up so the two siblings stay in sync.

```
interface UserItem {
  id: string;
  name: string;
}

function SearchBox({ value, onChange }: { value: string; onChange: (value: string) => void }) {
  return <input value={value} onChange={(e) => onChange(e.target.value)} placeholder="Search users..." />;
}

function UserList({ users, query }: { users: UserItem[]; query: string }) {
  const filtered = users.filter((u) => u.name.toLowerCase().includes(query.toLowerCase()));
  return (
    <ul>
      {filtered.map((u) => (
        <li key={u.id}>{u.name}</li>
      ))}
    </ul>
  );
}

// TODO: implement SearchableUserList({ users }) composing SearchBox and UserList
```

Your task:

1. Implement SearchableUserList, holding the query string in state at the parent level (not inside SearchBox or UserList).
2. Pass the query and its setter down to SearchBox and UserList as props so they stay in sync without either one owning the state itself.

MEDIUM

Fix Memo Object Identity Bug

A `UserCard` is wrapped in `React.memo` to avoid re-rendering when an unrelated counter elsewhere in the app updates. Console logs show it re-renders on every click anyway, even though the user data displayed never actually changes.

```
const UserCard = memo(function UserCard({ user }: { user: { name: string } }) {
  console.log("rendering", user.name);
  return <p>{user.name}</p>;
});

function App() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <button onClick={() => setCount(count + 1)}>{count}</button>
      <UserCard user={{ name: "Ada" }} />
    </div>
  );
}
```

Your task:

1. Explain why `memo()` fails to prevent `UserCard` from re-rendering here even though the displayed name never changes.
2. Fix App so the `user` object passed to `UserCard` keeps the same reference across renders when its contents haven't changed.

